

## M002 Week 4 — AND / OR

**Topic:** AND and OR Conditions — Smarter Decisions · **Course:** Maze Madness · **Level:** Advanced ·

**Time:** about 55 minutes

New this week: `or` (at least one true). Combine it with `and` in one maze.

- `and` → the action runs **only when both** conditions are true.
- `or` → the action runs **when at least one** condition is true.

`agent.detect(BLOCK, FORWARD)` is `True` when a block is ahead. Use `not` for an open path.

### 1 · Predict 🧠

Write what the agent does.

```
if agent.detect(BLOCK, FORWARD) and agent.detect(BLOCK, RIGHT):
    agent.turn(LEFT_TURN)
```

Wall ahead, open right. Does it turn left? Why?

|  |
|--|
|  |
|  |
|  |
|  |

```
if not agent.detect(BLOCK, FORWARD) or not agent.detect(BLOCK, DOWN):
    agent.move(FORWARD, 1)
```

Ahead blocked, below open. Does it move? Why?

|  |
|--|
|  |
|  |
|  |
|  |

```
while True:
    if agent.detect(BLOCK, FORWARD) and agent.detect(BLOCK, RIGHT):
        agent.turn(LEFT_TURN)
    elif not agent.detect(BLOCK, FORWARD) or not agent.detect(BLOCK, LEFT):
        agent.move(FORWARD, 1)
    else:
        agent.turn(RIGHT_TURN)
```

At a front-and-right corner, what runs? At an open junction, which branch runs?

|  |
|--|
|  |
|  |
|  |
|  |

## 2 · Trace the Truth 🔍

Write the branch that runs for each case.

```
if agent.detect(BLOCK, FORWARD) and agent.detect(BLOCK, RIGHT):  
    agent.turn(LEFT_TURN)  
elif not agent.detect(BLOCK, FORWARD):  
    agent.move(FORWARD, 1)  
else:  
    agent.turn(RIGHT_TURN)
```

| Front | Right | What runs? |
|-------|-------|------------|
| wall  | wall  |            |
| wall  | open  |            |
| open  | wall  |            |

|  |
|--|
|  |
|  |
|  |

### 3 · Spot and Fix Bugs 🐛

Rewrite each block. Explain the fix.

**Bug A** — Turn left **only** at a dead-end corner (wall ahead AND wall on right). It turns at any wall.

```
if agent.detect(BLOCK, FORWARD) or agent.detect(BLOCK, RIGHT):  
    agent.turn(LEFT_TURN)
```

Fixed code:

|  |
|--|
|  |
|  |
|  |
|  |

Why was it wrong? Why does your fix work?

|  |
|--|
|  |
|  |
|  |
|  |

**Bug B** — Move forward when **either** the path ahead OR the path below is clear.

```
if not agent.detect(BLOCK, FORWARD) and not agent.detect(BLOCK, DOWN):  
    agent.move(FORWARD, 1)
```

Fixed code:

|  |
|--|
|  |
|  |
|  |
|  |

Why was it wrong? Why does your fix work?

|  |
|--|
|  |
|  |
|  |
|  |

**Bug C** — At a fork, turn right if the right is open, otherwise turn left. The order is wrong, so it always turns left first.

```
if not agent.detect(BLOCK, LEFT):  
    agent.turn(LEFT_TURN)  
elif not agent.detect(BLOCK, RIGHT):  
    agent.turn(RIGHT_TURN)
```

Hint: check the right side first.

Fixed code:

|  |
|--|
|  |
|  |
|  |
|  |

Why was it wrong? Why does your fix work?

|  |
|--|
|  |
|  |
|  |
|  |

## 4 · Write the Code

Write a `while True` loop that:

- turns left when there is a wall ahead **and** a wall on the right,
- else moves forward when the path ahead **or** the path below is open,
- else turns right.

## 5 · Show Your Work 📷 🎥

Switch to your homework world. Solve the maze using **both** an **and** condition and an **or** condition.

Record **one video** — one take, no stopping (a phone is fine). Show these in order:

**1 · Your code.** Scroll through it. Say what each part does.

**2 · Your build.** Point the camera. Name the parts.

Fill the blanks:

Today I built \_\_\_\_\_.

I built it using this code: \_\_\_\_\_.

In this code I used \_\_\_\_\_.

The hardest part was \_\_\_\_\_.

That part was hard because \_\_\_\_\_.

The most fun part was \_\_\_\_\_.

Something new I learned was \_\_\_\_\_.

**Write your lines here, then say them in your video.**

|  |
|--|
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |

**Submit** ✓

Send this worksheet + your video to teacher on KakaoTalk.